

MODUL LENGKAP

ADMINISTRASI BASIS DATA

SQL Server Management Studio (SSMS)

Mencakup Materi:

- Materi 10 - SQL Server Agent & Otomasi Pekerjaan
- Materi 11 - Indeks dan Optimasi Query pada SQL Server
- Materi 13 - Monitoring Performa SQL Server

Universitas Muhammadiyah Yogyakarta

Mata Kuliah: Administrasi Basis Data

Tahun 2025

MATERI 10: SQL Server Agent & Otomasi Pekerjaan

A. Pendahuluan

Dalam lingkungan database modern, seorang administrator basis data (DBA) tidak mungkin melakukan seluruh pekerjaan secara manual setiap hari. Bayangkan jika setiap malam harus bangun jam 02.00 hanya untuk menjalankan backup database — tentu tidak efisien dan rawan lupa.

Aktivitas seperti backup database, pengecekan performa server, monitoring error, hingga pengiriman notifikasi harus dapat berjalan secara otomatis agar sistem tetap stabil dan aman sepanjang waktu.

Di sinilah SQL Server Agent berperan. SQL Server Agent adalah layanan (service) bawaan Microsoft SQL Server yang berfungsi sebagai "asisten otomatis" administrator database — menjalankan tugas-tugas berdasarkan jadwal atau kondisi tertentu, bahkan saat tidak ada operator manusia yang mengawasi.

Analogi Sederhana

Bayangkan SQL Server Agent seperti alarm dan timer otomatis di rumah Anda. Lampu menyala otomatis saat malam, AC mati otomatis saat pagi, dan oven memasak sendiri sesuai jadwal — semua tanpa Anda harus melakukannya manual setiap hari.

B. Apa itu SQL Server Agent?

SQL Server Agent adalah komponen bawaan SQL Server yang digunakan untuk:

- Menjadwalkan dan menjalankan pekerjaan (Jobs) secara otomatis
- Menjalankan skrip SQL, PowerShell, atau paket SSIS
- Monitoring kondisi server dan mengirim notifikasi
- Menjalankan maintenance database secara rutin
- Mendeteksi dan merespons kondisi tertentu (Alerts)

SQL Server Agent berjalan sebagai Windows Service bernama `SQLServerAgent`. Layanan ini harus dalam status `Running` agar seluruh fitur otomasi dapat berfungsi.

C. Komponen Utama SQL Server Agent

SQL Server Agent terdiri dari 5 komponen utama yang saling bekerja sama:

1. Jobs (Pekerjaan)

Job adalah unit utama otomatisasi dalam SQL Server Agent. Sebuah job merupakan kumpulan satu atau lebih langkah (steps) yang dieksekusi secara berurutan.

Nama Job	Fungsi	Jadwal Umum
Backup_Harian	Backup database setiap malam	Setiap hari jam 23:00
Cleanup_Log	Menghapus log transaksi lama	Setiap Minggu
Rebuild_Index	Optimasi performa indeks	Setiap Minggu
Update_Statistics	Memperbarui statistik query optimizer	Setiap hari
Monitor_Storage	Cek kapasitas disk server	Setiap 1 jam

2. Job Steps (Langkah Pekerjaan)

Setiap job terdiri dari satu atau lebih step. Setiap step dapat berisi berbagai jenis perintah:

- Transact-SQL (T-SQL) — perintah SQL biasa
- PowerShell — skrip Windows PowerShell
- Operating System (CmdExec) — perintah command line
- SSIS Package — paket SQL Server Integration Services
- Replication Distributor, Queue Reader, dll.

Setiap step dapat dikonfigurasi aksinya bila berhasil (on success) maupun gagal (on failure), misalnya lanjut ke step berikutnya, hentikan job, atau kirim notifikasi.

3. Schedules (Jadwal)

Schedule menentukan kapan sebuah job dijalankan. Jenis-jenis jadwal yang tersedia:

Jenis Schedule	Contoh Penggunaan
Recurring (Berulang)	Setiap hari jam 23:00, Setiap Senin jam 01:00
One Time (Sekali jalan)	Tanggal tertentu untuk migrasi data
On SQL Server Agent startup	Dijalankan saat layanan Agent dimulai
On CPU when idle	Dijalankan saat CPU tidak sibuk

4. Alerts (Peringatan Otomatis)

Alert adalah mekanisme yang memantau kondisi tertentu pada SQL Server dan mengambil tindakan otomatis ketika kondisi tersebut terjadi.

Jenis Alert	Pemicu	Contoh
SQL Server Event Alert	Error atau peringatan SQL Server	Error 9002: Log penuh
Performance Condition Alert	Counter performa melampaui batas	CPU usage > 90%
WMI Event Alert	Event Windows Management Instrumentation	Event log Windows tertentu

5. Operators (Penerima Notifikasi)

Operator adalah orang atau tim yang akan menerima notifikasi ketika terjadi alert atau job gagal. Notifikasi dapat dikirim melalui:

- Email (paling umum, menggunakan Database Mail)
- Net Send (pesan jaringan Windows)
- Pager (sudah jarang digunakan)

D. Arsitektur dan Alur Kerja SQL Server Agent

Berikut alur kerja SQL Server Agent secara lengkap:

Langkah	Komponen	Keterangan
1	Schedule	Jadwal yang telah dikonfigurasi mencapai waktunya
2	SQL Server Agent	Layanan Agent mendeteksi jadwal dan memulai job
3	Job	Job beserta seluruh step-nya dijalankan secara berurutan
4	Monitoring	Agent memantau status setiap step (berhasil/gagal)
5	Alert	Jika terjadi kegagalan, alert dipicu secara otomatis
6	Operator	Operator menerima notifikasi melalui email atau media lain

E. Jobs untuk Backup Otomatis

Salah satu penggunaan paling penting dari SQL Server Agent adalah otomasi backup database. Tanpa otomasi, risiko kehilangan data sangat besar karena bergantung pada kedisiplinan manusia.

Strategi Backup Bertingkat (3-2-1 Approach)

Jenis Backup	Fungsi	Jadwal Rekomendasi	Waktu Eksekusi
Full Backup	Backup seluruh isi database	Mingguan (Minggu malam)	Lama (berjam-jam)
Differential Backup	Backup perubahan sejak Full Backup terakhir	Harian (setiap malam)	Sedang
Transaction Log Backup	Backup setiap transaksi yang terjadi	Setiap 15-30 menit	Sangat cepat

Contoh Query T-SQL untuk Backup dalam Job Step

```
BACKUP DATABASE [NamaDatabase] TO DISK = N'C:\Backup\NamaDatabase_Full.bak' WITH
FORMAT, INIT, NAME = N'Full Backup', STATS = 10
```

F. Maintenance Plan

Maintenance Plan adalah fitur SQL Server Agent yang dirancang khusus untuk merencanakan dan menjalankan tugas-tugas pemeliharaan database secara otomatis dan terjadwal.

Tugas Maintenance	Tujuan	Frekuensi Rekomendasi
Rebuild Index	Mengoptimalkan struktur indeks yang terfragmentasi, mempercepat query	Mingguan
Reorganize Index	Defragmentasi ringan pada indeks (lebih cepat dari rebuild)	Mingguan
Update Statistics	Memperbarui data statistik untuk Query Optimizer	Harian / setelah perubahan data besar
Database Integrity Check	Mendeteksi kerusakan halaman data (corruption)	Mingguan
Shrink Database	Mengecilkan ukuran file database (gunakan dengan hati-hati)	Jarang, sesuai kebutuhan
Cleanup History	Menghapus riwayat job, backup, dan maintenance lama	Bulanan

G. SQL Server Agent untuk Keamanan Database

Selain untuk backup dan maintenance, SQL Server Agent juga berperan penting dalam keamanan sistem database. Agent dapat digunakan untuk mendeteksi aktivitas mencurigakan dan merespons secara otomatis.

Contoh skenario keamanan yang dapat diotomasi:

- Monitoring login gagal berulang (indikasi brute force attack)
- Deteksi INSERT massal dalam waktu singkat (indikasi injeksi data)
- Monitoring DELETE atau UPDATE tanpa klausa WHERE
- Mengirim alert email ke DBA ketika anomali terdeteksi
- Menjalankan job pengunci akun setelah beberapa kali login gagal

H. Keuntungan dan Risiko

Aspek	Dengan SQL Server Agent	Tanpa SQL Server Agent
Backup	Otomatis, terjadwal, konsisten	Manual, rawan lupa
Maintenance	Rutin, terencana	Tidak konsisten
Monitoring	Real-time, proaktif	Reaktif, terlambat
Notifikasi	Langsung via email	Harus cek manual
Efisiensi DBA	Tinggi, fokus pada hal strategis	Rendah, banyak pekerjaan rutin

MATERI 11: Indeks dan Optimasi Query pada SQL Server

A. Pendahuluan

Dalam sistem informasi modern, sebuah database dapat menyimpan ribuan hingga jutaan baris data. Ketika pengguna melakukan pencarian, SQL Server harus menemukan data tersebut secepat mungkin.

Tanpa optimasi, SQL Server terpaksa membaca setiap baris data satu per satu dari awal hingga akhir tabel — proses yang sangat lambat ketika tabel berisi jutaan record.

Analogi Buku

Bayangkan sebuah buku ensiklopedia setebal 1000 halaman tanpa daftar isi. Untuk mencari topik 'Zebra', Anda harus membuka halaman satu per satu dari halaman 1. Dengan daftar isi (indeks), Anda langsung menuju halaman yang tepat. Indeks database bekerja dengan prinsip yang sama.

B. Cara Kerja SQL Server Tanpa Indeks vs Dengan Indeks

Kondisi	Proses SQL Server	Nama Operasi	Performa
Tanpa Indeks	Membaca SETIAP baris dari baris pertama hingga terakhir	Table Scan / Clustered Index Scan	Lambat ($O(n)$)
Dengan Indeks	Langsung menuju lokasi data menggunakan struktur indeks	Index Seek	Sangat cepat ($O(\log n)$)

Contoh nyata: Tabel Mahasiswa dengan 100.000 baris. Mencari NIM '20220001':

- Tanpa indeks: SQL Server membaca 100.000 baris → lambat
- Dengan indeks pada kolom NIM: SQL Server langsung lompat ke data → sangat cepat

C. Struktur Data Indeks: B-Tree

SQL Server menyimpan indeks menggunakan struktur B-Tree (Balanced Tree). Struktur ini terdiri dari:

Level B-Tree	Nama Node	Fungsi
Paling atas (1 node)	Root Node	Titik awal pencarian; mengarahkan ke intermediate node
Tengah (bervariasi)	Intermediate Node	Menyimpan rentang nilai untuk memandu pencarian
Paling bawah	Leaf Node	Menyimpan nilai indeks aktual dan pointer ke data

Keunggulan B-Tree:

- Selalu seimbang — setiap leaf node berada di kedalaman yang sama
- Pencarian, penyisipan, dan penghapusan efisien: $O(\log n)$
- Sangat efektif untuk pencarian range (BETWEEN, >, <)

D. Jenis-Jenis Indeks di SQL Server

1. Clustered Index

Clustered Index adalah indeks yang menentukan urutan fisik penyimpanan data pada tabel. Data disimpan langsung sesuai urutan nilai indeks.

Karakteristik	Keterangan
Jumlah per tabel	Hanya SATU (karena menentukan urutan fisik data)
Lokasi data	Data dan indeks berada di struktur yang sama (leaf = data row)
Default	Otomatis dibuat pada PRIMARY KEY
Kecepatan range query	Sangat cepat (data sudah berurutan fisik)
Kecepatan INSERT/UPDATE/DELETE	Lebih lambat (harus menjaga urutan fisik)

Contoh T-SQL membuat Clustered Index:

```
-- Membuat Clustered Index pada kolom NIM tabel Mahasiswa
CREATE CLUSTERED INDEX IX_Mahasiswa_NIM
ON Mahasiswa (NIM ASC);
```

2. Non-Clustered Index

Non-Clustered Index adalah indeks terpisah dari data utama. Leaf node-nya menyimpan nilai indeks beserta pointer ke baris data (Row Locator).

Karakteristik	Keterangan
Jumlah per tabel	Bisa banyak (hingga 999 per tabel)
Lokasi data	Indeks dan data disimpan terpisah
Cocok untuk	Kolom yang sering digunakan di WHERE, JOIN, ORDER BY
Kecepatan baca	Cepat untuk pencarian spesifik
Kecepatan INSERT/UPDATE/DELETE	Lebih lambat karena perlu update indeks juga

Contoh T-SQL membuat Non-Clustered Index:

```
-- Membuat Non-Clustered Index pada kolom Nama
CREATE NONCLUSTERED INDEX IX_Mahasiswa_Nama
ON Mahasiswa (Nama ASC);
```

```
-- Non-Clustered Index dengan INCLUDE (Covering Index)
CREATE NONCLUSTERED INDEX IX_Mahasiswa_Nama_Cover
ON Mahasiswa (Nama ASC)
INCLUDE (NIM, Email, Jurusan);
```

3. Perbandingan Clustered vs Non-Clustered Index

Aspek	Clustered Index	Non-Clustered Index
Jumlah per tabel	Hanya 1	Hingga 999
Penyimpanan data	Data ada di leaf node	Pointer ke data di leaf node
Range query	Sangat cepat	Cepat
Lookup (bookmark lookup)	Tidak perlu	Kadang diperlukan
Fragmentasi	Mempengaruhi penyimpanan fisik	Hanya mempengaruhi indeks

4. Jenis Indeks Lainnya

Jenis Indeks	Fungsi	Kapan Digunakan
Unique Index	Memastikan tidak ada nilai duplikat pada kolom	Kolom yang harus unik selain PK (misal: email)
Composite Index	Indeks pada kombinasi 2+ kolom	Query yang filter berdasarkan beberapa kolom sekaligus
Filtered Index	Indeks pada subset data tertentu	Kolom dengan banyak nilai NULL atau subset tertentu

Jenis Indeks	Fungsi	Kapan Digunakan
Full-Text Index	Pencarian teks bebas pada kolom teks panjang	Pencarian kata dalam dokumen atau deskripsi panjang
Columnstore Index	Menyimpan data secara kolom (bukan baris)	Data warehouse dan query analitik (agregasi besar)

E. Optimasi Query

Membuat indeks saja tidak cukup. Query juga harus ditulis dengan efisien agar SQL Server dapat memanfaatkan indeks yang ada.

Teknik Optimasi Query

1. Hindari SELECT * — Pilih hanya kolom yang dibutuhkan

```
-- BURUK: Mengambil semua kolom
SELECT * FROM Mahasiswa WHERE Jurusan = 'Informatika';
```

```
-- BAIK: Pilih kolom yang diperlukan saja
SELECT NIM, Nama, Email FROM Mahasiswa WHERE Jurusan = 'Informatika';
```

2. Gunakan klausa WHERE dengan kolom terindeks

```
-- BURUK: Menggunakan fungsi pada kolom terindeks (membatalkan index)
SELECT * FROM Mahasiswa WHERE YEAR(TanggalLahir) = 2000;
```

```
-- BAIK: Gunakan range langsung
SELECT NIM, Nama FROM Mahasiswa
WHERE TanggalLahir BETWEEN '2000-01-01' AND '2000-12-31';
```

3. Hindari penggunaan LIKE dengan wildcard di awal

```
-- BURUK: Wildcard di awal (tidak bisa pakai indeks)
SELECT * FROM Mahasiswa WHERE Nama LIKE '%Ahmad%';
```

```
-- LEBIH BAIK: Wildcard hanya di akhir
SELECT * FROM Mahasiswa WHERE Nama LIKE 'Ahmad%';
```

4. Gunakan JOIN dengan benar

```
-- BAIK: Selalu JOIN pada kolom terindeks
SELECT m.Nama, n.NilaiAkhir
```

```
FROM Mahasiswa m
INNER JOIN Nilai n ON m.NIM = n.NIM -- NIM harus terindeks
WHERE m.Jurusan = 'Informatika';
```

F. Execution Plan

Execution Plan adalah peta yang menunjukkan langkah-langkah SQL Server dalam mengeksekusi sebuah query. Ini adalah alat utama untuk mengidentifikasi masalah performa query.

Cara melihat Execution Plan di SSMS:

1. Buka query di SSMS
2. Tekan Ctrl+M untuk mengaktifkan Actual Execution Plan
3. Jalankan query
4. Klik tab 'Execution plan' di bagian bawah

Operator	Arti	Implikasi Performa
Index Seek	SQL Server menggunakan indeks secara efisien	Sangat baik — indeks dimanfaatkan
Index Scan	SQL Server membaca seluruh indeks	Kurang baik — pertimbangkan covering index
Table Scan / Clustered Index Scan	Membaca seluruh tabel	Buruk — perlu indeks pada kolom filter
Key Lookup / Bookmark Lookup	Mencari data tambahan ke tabel utama	Perlu ditinjau — pertimbangkan INCLUDE index
Hash Match / Nested Loops	Metode JOIN yang digunakan	Hash Match untuk data besar, Nested Loops untuk data kecil
Sort	Pengurutan data	Mahal — pertimbangkan indeks pada kolom ORDER BY

G. Fragmentasi Indeks dan Cara Mengatasinya

Seiring berjalannya waktu dan banyaknya operasi INSERT, UPDATE, DELETE, indeks akan mengalami fragmentasi — kondisi di mana data tidak lagi tersimpan secara berurutan efisien.

Tingkat Fragmentasi	Tindakan yang Disarankan	Perintah T-SQL
< 5%	Tidak perlu tindakan	-

Tingkat Fragmentasi	Tindakan yang Disarankan	Perintah T-SQL
5% - 30%	Reorganize Index (lebih ringan, online)	ALTER INDEX IX_Nama ON Tabel REORGANIZE
> 30%	Rebuild Index (lebih efektif, bisa offline)	ALTER INDEX IX_Nama ON Tabel REBUILD

Query untuk memeriksa fragmentasi indeks:

```
SELECT
    object_name(ips.object_id) AS TableName,
    i.name AS IndexName,
    ips.avg_fragmentation_in_percent AS Fragmentasi
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED')
ips
INNER JOIN sys.indexes i ON ips.object_id = i.object_id
    AND ips.index_id = i.index_id
WHERE ips.avg_fragmentation_in_percent > 5
ORDER BY ips.avg_fragmentation_in_percent DESC;
```

MATERI 13: Monitoring Performa SQL Server

A. Pendahuluan

Seiring bertambahnya jumlah pengguna dan volume data, performa database menjadi semakin kritis. Database yang tidak dimonitor dengan baik akan mengalami degradasi performa yang berdampak langsung pada pengguna: aplikasi lambat, transaksi gagal, atau bahkan sistem down.

Monitoring performa bukan sekadar melihat apakah sistem berjalan — tetapi memahami BAGAIMANA sistem berjalan dan mengidentifikasi masalah SEBELUM berdampak pada pengguna.

B. Indikator Performa Utama SQL Server

Indikator	Deskripsi	Nilai Normal	Nilai Bermasalah
CPU Usage	Persentase penggunaan prosesor oleh SQL Server	< 70% rata-rata	> 90% terus-menerus
Memory Usage	RAM yang digunakan SQL Server (buffer pool)	Mendekati maksimum adalah normal	Page faults tinggi = RAM kurang
Disk I/O	Kecepatan baca/tulis ke disk	Tergantung jenis disk	Disk queue length > 2
Batch Requests/sec	Jumlah query diproses per detik	Tergantung workload	Menurun drastis = bottleneck
Page Life Expectancy	Berapa lama halaman data bertahan di memori	> 300 detik	< 300 detik terus-menerus
Lock Waits/sec	Jumlah tunggu akibat kunci data	Mendekati 0	Terus meningkat = blocking

C. Activity Monitor

Activity Monitor adalah alat monitoring bawaan SSMS yang menampilkan kondisi server secara real-time. Ini adalah alat pertama yang digunakan DBA ketika terjadi masalah performa.

Cara membuka Activity Monitor di SSMS:

5. Buka SQL Server Management Studio (SSMS)
6. Klik kanan pada nama server di Object Explorer
7. Pilih 'Activity Monitor'
8. Atau tekan Ctrl+Alt+A

Panel-Panel di Activity Monitor

Panel	Informasi yang Ditampilkan	Kegunaan
Overview	Grafik CPU, Waiting Tasks, Database I/O, Batch Requests	Gambaran umum kesehatan server
Processes	Daftar semua sesi/koneksi yang aktif	Melihat siapa yang terhubung dan apa yang sedang dikerjakan
Resource Waits	Jenis dan durasi tunggu sumber daya	Mengidentifikasi bottleneck (CPU, I/O, Lock, dll.)
Data File I/O	Aktivitas baca/tulis per file database	Mengidentifikasi database mana yang paling sibuk
Recent Expensive Queries	Query yang paling banyak menggunakan sumber daya	Menemukan query bermasalah yang perlu dioptimasi

Catatan Penting

Activity Monitor menggunakan sumber daya server untuk menampilkan data real-time. Di server produksi yang sibuk, refresh interval sebaiknya diatur lebih lambat (misal: 30-60 detik) agar tidak menambah beban server.

D. Performance Monitor (PerfMon)

Performance Monitor (PerfMon) adalah alat monitoring Windows yang dapat memantau tidak hanya SQL Server, tetapi seluruh sistem server termasuk hardware. PerfMon sangat berguna untuk memastikan apakah masalah performa berasal dari SQL Server atau dari keterbatasan hardware.

Cara membuka PerfMon:

9. Tekan Windows + R
10. Ketik 'perfmon' lalu Enter
11. Atau cari 'Performance Monitor' di Start Menu

Counter PerfMon Penting untuk SQL Server

Kategori Counter	Counter Spesifik	Threshold Perhatian
Processor	% Processor Time	> 80% sustained
Memory	Available MBytes	< 10% dari total RAM
PhysicalDisk	Avg. Disk Queue Length	> 2 per disk

Kategori Counter	Counter Spesifik	Threshold Perhatian
SQL Server: Buffer Manager	Page Life Expectancy	< 300 detik
SQL Server: SQL Statistics	Batch Requests/sec	Turun drastis dari baseline
SQL Server: Locks	Lock Waits/sec	> 0 secara konsisten
SQL Server: Memory Manager	Memory Grants Pending	> 0 secara konsisten

E. SQL Server Profiler

SQL Server Profiler adalah alat untuk merekam dan menganalisis semua event yang terjadi pada SQL Server. Profiler memungkinkan DBA melihat secara detail query apa yang dijalankan, siapa yang menjalankannya, berapa lama, dan berapa sumber daya yang digunakan.

Penting

SQL Server Profiler sudah deprecated (tidak dikembangkan lagi) sejak SQL Server 2012. Alternatif modernnya adalah Extended Events (XEEvents) yang lebih efisien. Namun Profiler masih tersedia dan berguna untuk debugging cepat.

Cara Menggunakan SQL Server Profiler

12. Di SSMS, buka menu Tools > SQL Server Profiler
13. Koneksikan ke server yang ingin dimonitor
14. Pilih Template (misalnya 'TSQL_Duration' untuk analisis performa)
15. Pilih event yang ingin direkam pada tab Events Selection
16. Klik Run untuk mulai merekam
17. Analisis hasil trace untuk menemukan query bermasalah

Event Penting yang Direkam Profiler

Event	Keterangan	Kegunaan
SQL:BatchCompleted	Query batch yang selesai dieksekusi	Analisis durasi dan sumber daya query
RPC:Completed	Stored procedure yang selesai dieksekusi	Analisis performa stored procedure
Deadlock graph	Informasi detail tentang deadlock	Investigasi dan mencegah deadlock
Audit Login / Logout	Event login dan logout pengguna	Audit keamanan dan monitoring akses

Event	Keterangan	Kegunaan
Exception	Error yang terjadi selama eksekusi	Debugging dan troubleshooting

F. Query Lambat (Slow Query)

Query lambat adalah salah satu masalah paling umum dalam performa database. Query yang seharusnya selesai dalam milidetik bisa memakan waktu detik bahkan menit.

Penyebab Query Lambat

- Tidak ada indeks pada kolom yang digunakan di WHERE atau JOIN
- Penggunaan SELECT * yang mengambil kolom tidak diperlukan
- Statistik indeks yang sudah usang (outdated statistics)
- Fragmentasi indeks yang tinggi
- Query dengan banyak JOIN yang tidak dioptimasi
- Penggunaan fungsi pada kolom di klausa WHERE (menghindari index seek)
- Parameter sniffing yang menghasilkan execution plan buruk
- Keterbatasan hardware (RAM, CPU, disk speed)

Cara Mengidentifikasi Query Lambat

Metode	Cara Penggunaan	Kelebihan
Activity Monitor	Lihat panel 'Recent Expensive Queries'	Cepat, real-time
SQL Server Profiler	Filter event dengan Duration > threshold tertentu	Detail lengkap, bisa direkam
DMV (Dynamic Management Views)	Query sys.dm_exec_query_stats	Sangat detail, berbasis data historis
Extended Events	Buat session untuk merekam slow query	Modern, efisien, rekomendasi Microsoft
Query Store	Fitur built-in sejak SQL Server 2016	Analisis regresi query, mudah digunakan

Contoh query menggunakan DMV untuk menemukan query paling lambat:

```
SELECT TOP 10
    qs.total_elapsed_time / qs.execution_count AS AvgDuration_microsec,
    qs.execution_count,
    qs.total_worker_time / qs.execution_count AS AvgCPU_microsec,
    SUBSTRING(qt.text, (qs.statement_start_offset/2)+1,
```

```

        ((CASE qs.statement_end_offset WHEN -1 THEN DATALENGTH(qt.text)
        ELSE qs.statement_end_offset END -
        qs.statement_start_offset)/2)+1) AS QueryText
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) qt
ORDER BY AvgDuration_microsec DESC;

```

G. Blocking

Blocking terjadi ketika sebuah transaksi mengunci (lock) baris atau halaman data, sehingga transaksi lain yang membutuhkan data yang sama harus menunggu sampai kunci dilepaskan.

Blocking adalah mekanisme NORMAL dalam database multi-user untuk menjaga konsistensi data (ACID). Masalah muncul ketika blocking berlangsung terlalu lama sehingga membuat antrian panjang.

Hirarki Lock di SQL Server

Mode Lock	Singkatan	Deskripsi
Shared Lock	S	Untuk operasi baca (SELECT). Bisa dimiliki banyak transaksi sekaligus
Exclusive Lock	X	Untuk operasi tulis (INSERT/UPDATE/DELETE). Hanya satu transaksi
Update Lock	U	Fase transisi antara Shared dan Exclusive
Intent Shared	IS	Indikasi akan mengunci di level yang lebih rendah dengan S
Intent Exclusive	IX	Indikasi akan mengunci di level yang lebih rendah dengan X

Query untuk melihat blocking yang sedang terjadi:

```

SELECT
    blocking_session_id AS BlockingSession,
    session_id AS BlockedSession,
    wait_type,
    wait_time / 1000.0 AS WaitTime_seconds,
    command,
    status
FROM sys.dm_exec_requests
WHERE blocking_session_id > 0
ORDER BY wait_time DESC;

```

Cara Mengatasi Blocking

- Buat transaksi sesingkat mungkin — jangan biarkan transaksi terbuka terlalu lama
- Akses tabel dalam urutan yang sama di semua transaksi
- Gunakan READ COMMITTED SNAPSHOT ISOLATION (RCSI) untuk mengurangi konflik baca-tulis
- Tambahkan indeks yang tepat agar operasi lebih cepat dan lock cepat dilepas
- Pantau dengan Activity Monitor dan hentikan session blocking jika diperlukan

H. Deadlock

Deadlock adalah kondisi yang lebih serius dari blocking. Deadlock terjadi ketika DUA atau lebih transaksi saling menunggu sumber daya yang dikunci oleh satu sama lain — sehingga tidak ada yang bisa maju (saling memblokir dalam lingkaran).

Contoh Skenario Deadlock

Transaksi A mengunci Tabel Mahasiswa, lalu mencoba mengunci Tabel Nilai. Transaksi B mengunci Tabel Nilai, lalu mencoba mengunci Tabel Mahasiswa. Keduanya saling menunggu → DEADLOCK!

Penanganan Deadlock oleh SQL Server

SQL Server secara otomatis mendeteksi deadlock menggunakan Lock Monitor yang berjalan setiap 5 detik. Ketika deadlock terdeteksi:

18. SQL Server memilih salah satu transaksi sebagai deadlock victim
19. Transaksi victim dibatalkan (rollback) dan error 1205 dikembalikan ke aplikasi
20. Transaksi lainnya dapat melanjutkan prosesnya

Cara Mencegah Deadlock

- Akses resource (tabel, baris) selalu dalam urutan yang sama di semua transaksi
- Gunakan transaksi yang singkat dan efisien
- Tambahkan indeks yang tepat untuk mempercepat akuisisi lock
- Gunakan READ COMMITTED SNAPSHOT ISOLATION
- Implementasikan retry logic di aplikasi untuk menangani error deadlock

Query untuk melihat riwayat deadlock:

```
-- Melihat riwayat deadlock dari SQL Server error log
SELECT XDL.value(' (//deadlock/process-list/process/@id) [1]',
'varchar(50)') AS Victim,
        XDL.value(' (//deadlock/process-list/process/@currentdb) [1]',
'varchar(50)') AS DB
FROM (
```

```

SELECT CAST(target_data AS XML) AS TargetData
FROM sys.dm_xe_session_targets t
INNER JOIN sys.dm_xe_sessions s ON t.event_session_address =
s.address
WHERE s.name = 'system_health'
AND t.target_name = 'ring_buffer'
) AS Data
CROSS APPLY
TargetData.nodes(' //RingBufferTarget/event[@name="xml_deadlock_report"] ')
AS XEventData(XDL);

```

I. Ringkasan Alat Monitoring dan Kapan Menggunakannya

Situasi	Alat yang Digunakan	Alasan
Server terasa lambat sekarang	Activity Monitor	Real-time, cepat, langsung dari SSMS
Ingin tahu query mana yang paling berat	Activity Monitor → Recent Expensive Queries	Menampilkan top consuming queries
Analisis performa CPU, RAM, disk secara menyeluruh	PerfMon	Monitoring level OS + SQL Server
Ingin rekam semua aktivitas query selama periode tertentu	SQL Server Profiler / Extended Events	Capture detail semua event
Investigasi deadlock yang sudah terjadi	Extended Events (system_health)	Menyimpan XML deadlock graph otomatis
Analisis query yang mengalami regresi performa	Query Store	Membandingkan plan historis
Melihat siapa yang sedang memblokir	Activity Monitor / sys.dm_exec_requests	Identifikasi blocking session

Kesimpulan dan Hubungan Antar Materi

Ketiga materi ini saling berkaitan erat dalam pengelolaan SQL Server yang optimal:

Materi	Inti Materi	Hubungan dengan Materi Lain
Materi 10: SQL Server Agent	Otomasi tugas-tugas berulang (backup, maintenance, monitoring)	Maintenance Plan menggunakan Agent untuk menjalankan Rebuild Index dan Update Statistics (Materi 11)
Materi 11: Indeks & Optimasi Query	Membuat pencarian data lebih cepat dan efisien	Fragmentasi indeks dipantau melalui Monitoring (Materi 13) dan diperbaiki otomatis oleh Agent (Materi 10)
Materi 13: Monitoring Performa	Memantau kondisi server dan mengidentifikasi masalah	Hasil monitoring menjadi dasar optimasi indeks (Materi 11) dan dapat dikonfigurasi sebagai Alert di Agent (Materi 10)

Filosofi DBA yang Baik

Seorang DBA yang baik adalah seperti pilot pesawat: selalu memantau instrumen (monitoring), memastikan mesin berjalan optimal (optimasi & indeks), dan memiliki sistem autopilot yang handal (SQL Server Agent). Kombinasi ketiganya memastikan database Anda terbang mulus tanpa turbulensi.